

1-mavzu. Python dasturlash tili asoslari.

Reja.

1. Python dasturlash tili asoslari, o'zgaruvchilar, ma'lumotlar turlari.
2. Arifmetik amallar, mantiqiy tip, mantiqiy ifodalar va solishtirish belgilari.
3. Funktsiyalar, lokal va global o'zgaruvchilar, modullar Istisnolarni boshqarish.

1. Python dasturlash tili asoslari, o'zgaruvchilar, ma'lumotlar turlari. Python dasturlash tilida jumladan boshqa dasturlash tillarida ham o'zgaruvchilar katta ahamiyatga ega hisoblanadi! **Python** dasturlash tilida boshqa dasturlash tillaridan farqli ularoq tip e'lon qilmaydi, ha aytgancha PHP da ham o'zgaruvchilar e'lon qilinmaydi!

O'zgaruvchi - Xotiraning nomlangan qismi bo'lib, o'zida ma'lum bir toifadagi qiymatlarni saqlaydi. O'zgaruvchining nomi va qiymatlari bo'ladi. O'zgaruvchining nomi orqali qiymat saqlanayotgan xotira qismiga murojaat qilinadi. Programma ishlashi jarayonida o'zgaruvchining qiymatini o'zgartirish mumkin. Har qanday o'zgaruvchini ishlatishdan oldin, uni e'lon qilish lozim.

Python dasturlash tilida o'zgaruvchini tipi e'lon qilinmaydi! Dastur kodini yozish jarayonida tipni o'zingiz hohlagan turga o'tkazishingiz mumkin. Buni esa keyingi darslarda ko'rib chiqamiz! Hozir siz bilan dasturlash tilini o'rganish jarayonida qiyinchilikka o'chramasligingiz uchun oddiysidan boshlaymiz!

```
x = 5
y = "Master Sherkulov"
print(x)
print(y)
5
Master Sherkulov
```

O'zgaruvchilarni biron bir aniq turi bilan e'lon qilish shart emas va ular o'rnatilgandan keyin ham turini o'zgartirishi mumkin.

```
x = 4 # x integer turiga mansub
x = "Master Sherkulov" # x string tipi o'zgartirildi. oldingi qiymat yuqoldi!
print(x)
Master Sherkulov
```

O'zgaruvchilar nom qo'yish!

O'zgaruvchi qisqa nomga ega bo'lishi mumkin (masalan, x va y) yoki ko'proq tavsiflovchi nom (yosh, karnay, master va h.k). Python o'zgaruvchilar uchun qoidalar:

- O'zgaruvchilar **[a-Z]** harf bilan yoki **"_"** pastga belgi bilan boshlanishi zarur!
- O'zgaruvchilar nomi raqam bilan boshlanmaydi!
- O'zgaruvchi **[a-Z]** harf bilan yoki **"_"** pastga belgi bilan boshlanib raqamlar ham aralashtirsa bo'ladi. (*_ali123*, *son90* va *h.k*)
- O'zgarvchi nomida katta va kichik harflar ahamiyatga ega bo'lib misol uchun **Aka** va **aka** ikki xil o'zgaruvchi nomi hisoblanadi! (Aka, aka, AkA, aKA)

PYTHON tilida turli tipdagi o'zgaruvchilardan foydalanish mumkin, shu sababli, har bir tipdagi o'zgaruvchilar qanday tavsiflanishini bilish zarur. PYTHON tilida bitta o'zgaruvchini dastur bajarilishi davomida satr yoki son sifatida ishlatish mumkin. Shu bilan birga PYTHON tilida o'zgaruvchilar bilan ishlanganda oshkor ko'rsatilishi mumkin bo'lgan asosiy ma'lumotlar tiplari to'plami

mavjud.

- Butun (integer) sonlar – Sonning kasr bo'lmagan son bo'lib, ularda sonning asosi (10 lik), o'n oltilik (asosi 16-prefiksga ega) yoki sakkizlik (asosi 8-prefiksli) sanoq sistemalar ko'rsatiladi.

- Siljувchi vergulli (float) sonlar – sonning kompyuterda amalga oshiriladigan eksponentsiol yozuvi. Xuddi shuningdek, “ikkilangan aniqlikga” ega bo'lgan son ham mavjud.

- Satr (string) – simvollar ketma – ketligidan iborat bo'lib, unda har bir simvol bir bayt o'lchamdan, toki maksimal uzunlik 216 gacha bo'lgan joyni egallaydi.

Yakka qavslarga olingan satrlar literallar sifatida qoraladi, ayni paytda ikkilangan qavslar ichidagi satrlar esa (maxsus belgilar, o'zgaruvchilarning qiymatlari va shu kabilar) sifatida talqin qilinadi.

- Bul (boolean) tipi - Mantiqiy ifoda bo'lib, uning qiymati faqat rost (True) yoki yolg'on (False) dan iborat.

- Kompleks (complex) tipi – Sonning birinchi argument sifatida haqiqiy qism, ikkinchi argument sifatida mavhum qism uzatiladi.

- Massiv (array) – bir nechta qiymatlarning tartiblashtirilgan xaritasi bo'lib, undagi kalitlar qiymatlarga mos keladi. Kalitlar – bu indeks nomerlari (ular so'zsiz tushuniladi) yoki aniq ko'rsatilgan nishonlar. misol

- Ob'ekt – bu berilganlarning xossalarini saqlovchi va berilganlarni qayta ishlash metodlaridan iborat bo'lgan sinf.

- Resurs – tashqi resursga havola bo'lib, ular maxsus funksiyalar tomonidan yaratiladi va saqlanadi.

- NULL – qiymatga ega bo'lmagan o'zgaruvchi. Bu o'zgaruvchi, shakllantirilmagan bo'ladi (unga hech bir qiymat berilmagan bo'ladi), agar unga NULL o'zgarmasi ta'minlangan bo'lsa yoki unset() funksiyasi yordamida bekor qilinmagan bo'lsa.

6.2. Arifmetik amallar, mantiqiy tip, mantiqiy ifodalar va solishtirish belgilari.

Pythonda, boshqa dasturlash tillaridagi kabi o'zgaruvchilar aniq bir turdagi berilganlarni g' saqlash uchun xizmat qiladi. Pythonda o'zgaruvchilar alfavit belgilari yoki tag chizig'i belgisi bilan boshlanishi va tarkibi son, alfavit belgilari, tag chizig'i belgilaridan iborat bo'lishi, ya'ni bir so'z bilan aytganda identifikator bo'lishi kerak. Bundan tashqari o'zgaruvchi nomi Pythonda ishlatiladigan kalit so'zlar nomi bilan mos tushmasligi shart. Masalan, o'zgaruvchi nomi *and, as, assert, break, class, continue, def,*

del, elif, else, except, False, finally, for, from, global, if, import, in, is, lambda, None, nonlocal, not, or, pass, raise, return, True, try, while, with, yield kabi kalit soʻzlar nomi bilan mos tushishi mumkin emas.

Masalan, oʻzgaruvchini aniqlash (hosil qilish) quyidagicha amalga oshiriladi:

Copy

```
a = 14
name = "SDY"
```

Yuqorida `a` va `name` oʻzgaruvchilari yaratildi va ularga qiymat berildi. Shuni alohida taʼkidlash kerakki, Pythonda oʻzgaruvchini dastlab eʼlon qilish degan tushuncha mavjud emas (masalan: python tilida `int a` kabi oʻzgaruvchi eʼlon qilinadi), balki oʻzgaruvchi kiritiladi va unga qiymat beriladi (masalan: `a=14`). Berilgan qiymatga koʻra interpretator oʻzgaruvchining turini aniqlaydi. Pythonda oʻzgaruvchilarni nomlashning ikki turi: “camel case” va “underscore notation” turlaridan foydalanish tavsiya qilingan.

“camel case” turida oʻzgaruvchiga nom berilganda, agar oʻzgaruvchi nomi alohida soʻzlar birikmasidan tashkil topgan boʻlsa, ikkinchi soʻzdan boshlab har bir soʻzning birinchi harfi katta harfda (katta registr) boʻlishi talab qilinadi. Masalan:

Copy

```
firstName = "Saidov"
```

“underscore notation” turida esa soʻzlar orasiga tag chizigʻi “_” belgisi qoʻyiladi. Masalan:

Copy

```
first_name = "Saidov"
```

Oʻzgaruvchilar biror bir turdagi berilganlarni saqlaydi. Pythonda bir necha xildagi berilganlar turlari mavjud boʻlib, ular odatda toʻrtta guruhga ajratiladi: sonlar, ketma-ketliklar, lugʻatlar va toʻplamlar:

`bool` (*boolean*) – True va False mantiqiy qiymatlar uchun;

`int` – butun sonlar uchun, butun turdagi songa kompyuter xotirasida 4 bayt joy ajratiladi;

`float` – suzuvchan nuqtali sonlar (haqiqiy sonlar) uchun, haqiqiy sonlarni saqlash uchun kompyuter xotirasidan 8 bayt joy ajratiladi;

`complex` – kompleks sonlar uchun;

`str` – satrlar uchun, Python 3.x versiyasidan boshlab satrlar bu- Unicode kodirovkasidagi belgilar ketma-ketligini ifodalaydi;

`bytes` – 0-255 diapazondagi sonlar ketma ketligi uchun

`byte array` – baytlar massivi uchun;

`list` – ro‘yxatlar uchun;

`tuple` – kortejlar uchun;

`set` – tartiblanmagan unikal ob‘ektlar kolleksiyasi uchun;

`frozen set` – set singari, faqat u o‘zgartirilishi mumkin emas (immutable);

`dict` – lug‘atlar uchun. Har bir element kalit so‘z va qiymat juftligi ko‘rinishida ifodalaniladi.

Python –dinamik turlarga ajratuvchi dasturlash tili hisoblanadi. Yuqorida aytib o‘tilganidek, Pythonda o‘zgaruvchi turi unga yuklangan qiymat orqali aniqlanadi. Agarda o‘zgaruvchiga bittalik („“) yoki ikkitalik (“”) qo‘shtirnoq yordamida satr yuklansa, o‘zgaruvchi `str` turiga ega bo‘ladi, agarda o‘zgaruvchiga butun son yuklansa – `int`, haqiqiy son yuklansa (masalan: 3.14) yoki eksponentsial ko‘rinishdagi qiymat yuklansa (masalan: 11e-1) u `float` turiga ega bo‘ladi. Masalan:

Copy

```
user_id = 234 # int
x = 1.2e2 # = 1200.0 float
y = 6.7e-3 # = 0.0067 float
z = 1.223 # float
user_password = "sdy123" # str
b = True # bool
```

Pythonda haqiqiy (float) turidagi o‘zgaruvchilar [-10308 , +10308] oraliqdagi sonlar bilan hisoblash ishlarini amalga oshirsa bo‘ladi, lekin faqat 18 ta raqamlar ketma-ketligi ko‘rinadi (konsol ekraniga chiqarilganda). Ixtiyoriy katta yoki kichik sonlarni o‘zgaruvchidagi ifodasi 18 ta belgidan oshib ketsa, u holda eksponentsial orqali yaxlitlab ifodalanadi.

Shuni ham ta’kidlash kerakki, Pythonda o‘zgaruvchiga yangi qiymat berish orqali uning turi o‘zgartirilishi mumkin. Masalan:

Copy

```
age = 17 # int
print(age)

age = "o'n etti" # str
print(age)
```

Ushbu dasturda dastlab `age = 17` ifodasi orqali `age` o‘zgaruvchisi `int` turiga ega edi. Keyingi `age = "o'n etti"` ifoda bilan uning turi `str` turiga o‘zgartirildi. Bundan keyingi jarayonlarda `age` o‘zgaruvchisi eng ohirgi yuklangan qiymat turiga mos bo‘ladi.

O'zgaruvchilarning turini aniqlashda `type()` – funksiyasidan foydalaniladi.

Masalan:

Copy

```
age = 17 print(type(age))
```

```
age = "o'n etti"
```

```
print(type(age))
```

Konsol ekranidagi natija:

```
<class 'int'>
```

```
<class 'str'>
```

Python dasturlash tilida quyidagi ma'lumot turlari mavjud.

Code	Result
\'	Single Quote
\\	Backslash
\n	Yangi qator
\r	Carriage Return
\t	Tab
\b	Backspace
\f	Form Feed
\ooo	Octal value
\xhh	Hex value

Python dasturlash tilida **type()** Funktsiya yordamida istalgan ob'ekt ma'lumotlarini olishingiz mumkin. misol uchun: `print(type(attribute))`

```
n = 13
```

```
print(type(n))
```

```
<class 'int'>
```

Maxsus ma'lumot turini o'rnatish

Agar ma'lumotlar turini ko'rsatmoqchi bo'lsangiz, quyidagi konstruktor funksiyalaridan foydalanishingiz mumkin:

x = str("MasterSherkulov")	str
x = int(13)	int
x = float(13.15)	float
x = complex(13j)	complex
x = list(("KI", "IT", "913-15"))	list
x = tuple(("KI", "IT", "913-15"))	tuple
x = range(13)	range
x = dict(name="MasterSherkulov", age=24)	dict
x = set(("KI", "IT", "913-15"))	set
x = frozenset(("KI", "IT", "913-15"))	frozenset
x = bool(13)	bool
x = bytes(13)	bytes
x = bytearray(13)	bytearray
x = memoryview(bytes(13))	memoryview

Pythonda sonlar ustuda amal bajaruvchi ichki funksiyalar juda ko‘p. Xususan, *int()* va *float()* funksiyalari argument sifatida berilgan qiymatlarni mos ravishda butun va haqiqiy sonlarga o‘girish uchun ishlatiladi. Masalan:

Copy

```
a = int(10.0)
print(A) # 10
```

```
b = float("12.3")
print(B) # 12.3
```

```
c = str(12)
print(D) # "12"
```

```
d = bool(D)
print(E) # True
```

Turga keltirish funksiyalari odatda konsol ekranidan kiritilgan qiymatlarni kerakli turga o‘girish (chunki konsoldan kiritilgan ixtiyoriy qiymat *str* turiga tegishli bo‘lishi oldindan qabul qilingan) va ifodalarda bir turdan ikkinchi turga keltirish zarur bo‘lgan hollarda ishlatiladi. Masalan:

Copy

```
son1 = 3
son2 = input()
print(son1 + son2)
```

Ushbu dastur bajarilishi jarayonida turlar mos kelmasligi (*TypeError: unsupported operand type(s) for +: 'int' and 'str'*) to'g'risidagi xatolik ro'y bergani haqidagi xabarni chiqaradi. Turga keltirish funksiyasidan foydalanib quyidagicha dasturni qayta yozamiz:

Copy

```
son1 = 3
son2 = "12"
res = son1 + int(son2) print(res) #15
```

Ushbu dastur konsol ekraniga 15 degan javobni chiqaradi. Demak turga keltirish amali `int()` joyida to'g'ri qallanilgan.

`float()` – turga keltirish funksiyasi ham xuddi yuqoridagidek ishlatiladi. Faqat suzuvchan nuqtali haqiqiy sonlar ustida amallar bajarilganida natija har doim ham biz kutganday bo'lmaydi. Masalan:

Copy

```
son1 = 4.01
son2 = 5
son3 = son1 / son2
print(son3) #0.8019999999999999
```

Ushbu dasturda javob 0.802 chiqishi kerak edi, lekin uni javobi yuqoridagi misolda ko'rinib turganidek 0.8019999999999999 qiymatni ekranga chiqaradi. Bu qiymat hato emas. Haqiqiy sonlarning kompyuter xotirasida saqlanish formati butun sonlarnikidan farqlanadi. Shu sababli suzuvchan nuqtali sonlar qiymati taqriban saqlanadi (absolyut xatolik inobatga olmasa ham bo'ladigan darajada kichik). Shuning uchun haqiqiy sonlarni yahlitlash uchun `round()` funksiyasidan foydalaniladi.

Copy

```
son1 = 4.01
son2 = 5
son3 = round (son1 / son2,4)
print(son3) #0.802
```

`round(a,n)` funksiyasi ikkita parametr qabul qilib, dastlabkisi yahlitlanishi kerak bo'lgan qiymat, ikkinchisi verguldan keyin nechta belgi aniqlikda chiqarilishi kerakligini anglatuvchi son.

6.3. Funksiyalar, lokal va global o'zgaruvchilar, modullar Istisnolarni boshqarish.

Funksiyadan tashqarida yaratilgan o'zgaruvchilar (yuqoridagi barcha misollarda bo'lgani kabi) global o'zgaruvchilar deb ataladi.

Global o'zgaruvchilar funksiyalar ichida ham, tashqarisida ham hamma tomonidan foydalanishi mumkin.

Funksiya tashqarisida o'zgaruvchi yaratish va undan funksiya ichida foydalanish:

Copy

```
x = "awesome"
def myfunc():
    print("Python is " + x)
myfunc()
```

Agar funksiya ichida ham global o'zgaruvchi nomi bir xil nomdagi o'zgaruvchi yaratsangiz, bu o'zgaruvchi lokal o'zgaruvchi bo'ladi va u faqat funksiya ichida

ishlatilishi mumkin. Xuddi shu nomdagi global o'zgaruvchi avvalgidек, global bo'lib qoladi va qiymatini yo'qoymaydi.

Funksiya ichida global o'zgaruvchi nomi bilan bir xil bo'lgan o'zgaruvchi yaratish:
Copy

```
x = "awesome"
def myfunc():
    x = "fantastic"
    print("Python is " + x)
myfunc()
print("Python is " + x)
```

Global kalit so'zi

Odatda, funksiya ichida o'zgaruvchi yaratganingizda, bu o'zgaruvchi local holatda bo'ladi, va faqatgina funksiya ichida ishlatilishi mumkin.

Funksiya ichida global o'zgaruvchi yaratish uchun, global kalit so'zidan foydalanishingiz mumkin.

Agar siz global kalit so'zdan foydalansangiz, o'zgaruvchi global qamrovga tegishli bo'ladi:

Copy

```
def myfunc():
    global x
    x = "fantastic"
myfunc()
```

```
print("Python is " + x)
```

Bundan tashqari, global o'zgaruvchi qiymatini funksiya ichida o'zgartirmoqchi bo'lsangiz, global kalit so'zdan foydalaninE.

Global o'zgaruvchini funksiya ichida o'zgartirish uchun, o'zgaruvchiga global kalit so'zi orqali murojaat qilish:

Copy

```
x = "awesome"
def myfunc():
    global x
    x = "fantastic"
myfunc()
print("Python is " + x)
```